

MODULE 1

PYTHON PROGRAMMING

1BPLC105B / 205B

FOUNDATIONS • PROBLEM SOLVING • PYTHON BASICS • DEBUGGING

The Python Programming Journey

Think → Design → Code → Run → Understand



Course Notes

Kiran Mudaraddi

Learn the idea • Visualize the logic • Write the code • Test the result

Table of Contents

Part A — Programming Fundamentals

1. What is a Program?
2. Types of Programming Languages
3. Problem-Solving Tools (Algorithm, Flowchart, Pseudocode)
4. Introduction to Python (+ Compiler vs Interpreter, Popular IDEs)
 - Indentation — Python's Block Structure

Part B — Variables, Expressions & Statements

5. Values, Data Types & Strings (+ Escape Sequences)
6. Variables · Names & Keywords · Statements (+ Mutable vs Immutable)
7. Expressions, Operators & Operands (+ Comparison & Logical Operators)
8. Type Converters (+ Built-in Functions Quick Reference)
9. Order of Operations (+ Complete Precedence Chart)
10. String Operations · input() · Composition
11. The Modulus Operator

Part C — Iteration

12. Assignment & Updating Variables
13. The for Loop (+ range() in Detail) · The while Loop
14. Collatz Sequence · Tables · break / continue / pass
15. Paired Data & Nested Loops

Part D — Functions

16. Functions with Arguments · Return Values

Part E — Practice Programs (Solved Examples)

Swap Variables · Temperature Converter · Simple Interest · Leap Year
Multiplication Table · Star Pattern · Factorial · Fibonacci
Prime Check · Reverse Digits · Senior Citizen Check · Mean/Variance/StdDev
Digit Frequency · Armstrong Number · GCD · Palindrome Check · Menu-Driven Calculator

Appendix

- A. Glossary of Key Terms
- B. Do's and Don'ts for Writing Good Python Code
- C. Quick-Revision Summary

Part A — Programming Fundamentals

1. What is a Program?

Program: A set of instructions given to a computer to perform a specific task.

A program is a sequence of instructions that tells the computer how to perform a computation.

Types of computation

Type	Description	Example
Mathematical computation	Solving equations, finding roots of a polynomial	Quadratic equations
Symbolic computation	Searching/replacing text, compiling another program	Find & Replace in a document

Basic building blocks of every program

The Core Pattern of a Program

Every program transforms data into a useful result.

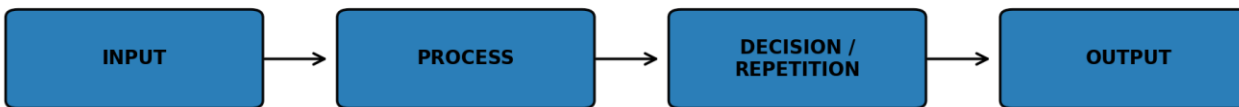


Figure: Input → Process → Decision/Repetition → Output

- Input – Get data from a keyboard, file, or other device.
- Output – Display data on the screen or send it to a file.
- Math – Perform basic operations like addition, subtraction, multiplication, division.
- Conditional execution – Check conditions and run code accordingly (if–else).
- Repetition – Perform actions repeatedly (loops).

Figure: Every program, however complex, is built from this same Input → Process → Output pattern.

2. Types of Programming Languages

Language Type	Made of	Structure	Portability	Translator
Machine-Level	0's and 1's (binary)	Unstructured	Not portable	None
Assembly-Level	Mnemonics (ADD, SUB, MUL, DIV)	Unstructured	Not portable	Assembler
High-Level	English-like statements	Structured	Portable	Compiler/Interpreter

Examples of high-level languages: Python, Java, C, C++

Examples of low-level languages: Assembly language, Machine code

Why high-level languages are preferred

- Easier and faster to write programs — saves development time.
- Shorter and more readable — fewer errors.
- Portable — the same program runs on different computers/hardware.

- Easier to debug and maintain over the program's lifetime.
- Rich ecosystem of libraries, IDEs, and community support.

Trade-offs of high-level languages

- Slightly slower execution than machine/assembly code (translation overhead).
- Less direct control over hardware resources (memory, registers).
- Requires a compiler/interpreter installed on the target machine.

3. Problem-Solving Tools

From Problem to Python



Figure: A practical path from a problem to Python code

Problem solving: is the process of analyzing a problem, designing a solution, and implementing it in code.

The three main problem-solving tools are: Algorithm, Flowchart, and Pseudocode.

PROBLEM → ALGORITHM → FLOWCHART → PSEUDOCODE → PYTHON CODE

3.1 Algorithm

A step-by-step, finite sequence of instructions designed to solve a specific task.

Example — Add two numbers:

```

Step 1: Start
Step 2: Input n1 and n2
Step 3: Compute sum ← n1 + n2
Step 4: Output sum
Step 5: Stop
  
```

Characteristics of a good algorithm

Characteristic	Meaning
Finiteness	Must terminate after a finite number of steps
Definiteness	Each step must be clear and unambiguous
Input	Takes zero or more inputs
Output	Produces at least one output
Effectiveness	Each step can be carried out using basic operations

Advantages: Language-independent, easy to debug logic before coding, useful for communicating a solution to others.

Disadvantages: Time-consuming for complex problems; no visual appeal; doesn't show branching as clearly as a flowchart.

3.2 Flowchart

A diagrammatic representation of a process or algorithm using standard symbols.

Symbol	Shape	Meaning
Start / Stop	Oval	Beginning or end of the flowchart
Input / Output	Parallelogram	Reading or displaying data
Computation / Assignment	Rectangle	Processing or calculations
Decision	Diamond / Rhombus	Represents a yes/no or true/false branching point (if-else)
Flow lines	Arrows	Direction of the process flow

Example — flowchart to add two numbers:

```

Start
↓
Input n1, n2
↓
sum ← n1 + n2
↓
Output sum
↓
Stop

```

Advantages: Visual and easy to understand at a glance; great for explaining logic to non-programmers; helps spot logical gaps early.

Disadvantages: Becomes cluttered for large or complex programs; redrawing is tedious when logic changes.

3.3 Pseudocode

A high-level description of an algorithm, written in simple English mixed with basic programming structures. It bridges the gap between an algorithm and actual code.

Advantages: Closer to real code than a flowchart, so easier to convert directly into a program; no strict syntax rules.

Disadvantages: Not standardized (varies from person to person); not executable, so it can't be tested directly.

Memory tip: Algorithm = language for the logic. Flowchart = language for the eyes. Pseudocode = language for the fingers (closest to real code).

Side-by-side comparison

Aspect	Algorithm	Flowchart	Pseudocode
Form	Numbered text steps	Diagram with symbols	English-like code
Best for	Explaining logic in words	Visualizing flow & branching	Drafting before real coding
Ease of editing	Easy	Tedious (must redraw)	Easy
Closeness to code	Low	Low-Medium	High

4. Introduction to Python

Python was created by Guido van Rossum in the late 1980s while working at the CWI (Centrum Wiskunde & Informatica) — the National Research Institute for Mathematics and Computer Science, in the Netherlands. He wanted to design an easy, English-like language that would be simple to read and write.

Timeline of major releases

Year	Version	Notes
1991	Python 0.9.0	First public release
1994	Python 1.0	Gained popularity among developers
1997	Python 1.5	—
2000	Python 2.0	Introduced list comprehensions, garbage collection
2008	Python 3.0	Major redesign, not fully backward-compatible
2022	Python 3.11	Performance and feature improvements

Features of Python

- Easy to learn, read, and write.
- Interpreted language (no separate compilation step needed).
- Object-oriented programming language.
- Free and open source for individuals and businesses.
- Supports multiple platforms — highly portable, integrates with any project.
- Has a huge standard library with well-documented modules and frameworks.
- Versatile, extensible, and flexible.
- Supports GUI (Graphical User Interface) development.
- Dynamically typed language.
- Backed by a huge, active community that keeps improving it.

How Python Code Runs: Interpreter vs Compiler

How Python Code Runs

A practical view of Python's source → bytecode → PVM execution path.

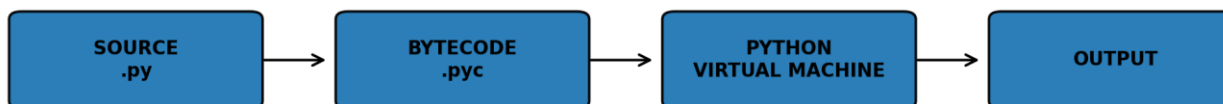


Figure: Python source → bytecode → Python Virtual Machine → output

A compiler translates the entire source code into machine code before execution (as in C/C++), producing a standalone executable. An interpreter translates and executes code line by line, without producing a separate executable file first.

Python is technically a hybrid: your .py source is first compiled into an intermediate low-level form called bytecode (.pyc), and then this bytecode is executed line by line by the Python Virtual Machine (PVM) — which is why Python is usually described simply as an 'interpreted' language.

SOURCE CODE (program.py) --compiled--> BYTECODE (.pyc) --interpreted--> PYTHON VIRTUAL MACHINE (PVM) --> OUTPUT

Aspect	Compiled (e.g., C, C++)	Interpreted (e.g., Python)
Translation	Whole program translated once, ahead of execution	Translated & executed line-by-line (via bytecode + PVM)
Execution speed	Faster (direct machine code)	Slower (extra translation layer at runtime)
Error detection	Most errors caught before running, at compile time	Errors often surface only when that line executes
Portability	Executable is platform-specific	Same source code/bytecode runs anywhere Python is installed

Popular Python IDEs / Tools

Tool	Good for
IDLE	The default editor bundled with Python — simple, lightweight, great for beginners
PyCharm	Full-featured professional IDE with debugging, refactoring, and project management
VS Code	Lightweight, highly extensible editor with excellent Python plugin support
Jupyter Notebook	Cell-based interactive coding — ideal for data science and step-by-step experimentation
Google Colab	Jupyter-style notebooks that run free in the browser — no local installation needed
Thonny	Beginner-friendly IDE with a built-in step-by-step variable visualizer

Advantages of Python

- Simple, readable, English-like syntax → fast to learn and code in.
- Huge standard library + third-party packages (NumPy, Pandas, Django, TensorFlow, etc.).
- Cross-platform: same code runs on Windows, Linux, macOS.
- Great for rapid prototyping, automation, and scripting.
- Strong community support and abundant learning resources.

Real-world applications of Python

Domain	Examples
Web Development	Django, Flask-based websites
Data Science & Analytics	Pandas, NumPy, Matplotlib
Artificial Intelligence / ML	TensorFlow, PyTorch, scikit-learn
Automation / Scripting	Task automation, web scraping
Game Development	Pygame
Desktop GUI Applications	Tkinter, PyQt
Networking & Cybersecurity	Scapy, penetration-testing tools

Domain	Examples
Embedded Systems / IoT	MicroPython, Raspberry Pi projects

1.1 The Python Programming Language

Programming is the process of writing instructions (code) that a computer can execute to perform tasks like calculation, decision-making, and data processing. Python is a high-level, user-friendly programming language, which is why it is so popular.

The Python Interpreter

The interpreter is the engine that translates and runs Python code. There are two ways to use it:

1. Immediate (Interactive) Mode — code is typed directly into the interpreter and the result is shown immediately.

```
>>> 2 + 2
4
```

The >>> is called the Python prompt — it signals that Python is ready to take instructions. Good for quick testing and trying out small snippets.

2. Script Mode — code is saved in a file (a script) and executed later, using the .py extension.

Advantages of Script Mode: code can be saved and reused; easy to debug and maintain; can be printed or shared; supported by text editors and IDEs.

Limitation of Immediate Mode: nothing is saved once the session ends — not suitable for large or reusable programs.

Indentation — Python's Block Structure (Important!)

Unlike C, C++, or Java, which use curly braces { } to mark a block of code, Python uses indentation (whitespace at the start of a line) to define blocks — for if statements, loops, and functions. This is not a style choice; it is part of Python's syntax.

```
if 5 > 2:
    print("Five is greater than two!") # indented → inside the if-block
print("This runs regardless") # not indented → outside the if-block
```

Common beginner error: Inconsistent indentation (mixing tabs and spaces, or wrong indent levels) raises an `IndentationError`. Always use 4 spaces per indent level, consistently.

1.2 What is Debugging?

Debugging is the process of finding and fixing errors (bugs) in a program. A bug is a mistake that causes the program to behave unexpectedly.

Three Types of Errors

Error type	When it occurs	Example
Syntax Error	Python doesn't understand the code because grammar rules are broken; Python stops immediately	print "Hello" (missing parentheses)
Runtime Error	Occurs while the program is running; also called an exception	Dividing by zero: 5/0
Semantic Error	Program runs without crashing but produces the wrong output because the logic is flawed	Using + when - was intended

Note: Semantic errors are the hardest to detect since no error message is shown — you must analyze the output and trace back through the logic to find the mistake.

Experimental Debugging

Experimental Debugging Cycle

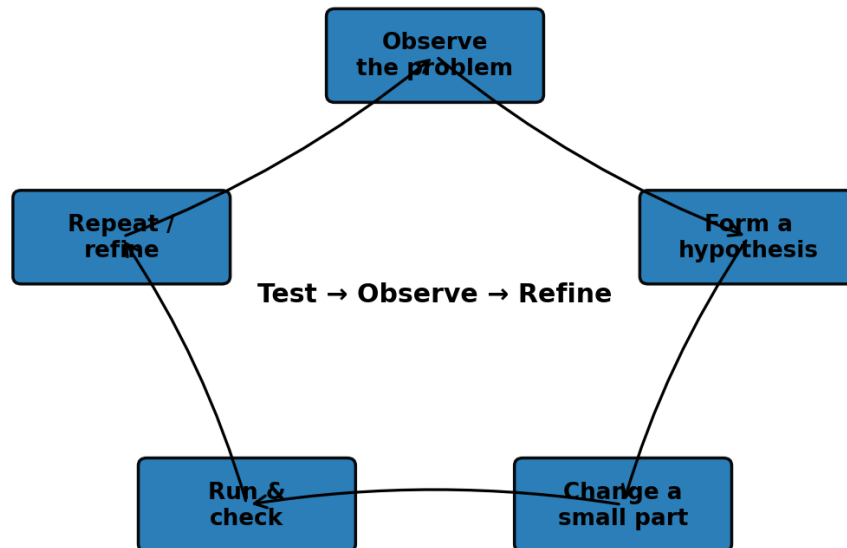


Figure: Experimental debugging as an iterative cycle

Debugging is one of the most important skills in programming. It:

- Improves logical thinking and problem-solving skills.
- Resembles detective work — piecing together clues to find the culprit (the bug).
- Resembles the scientific method — form a hypothesis, test it experimentally, refine based on the outcome.

Practical debugging tips (bonus)

- 1. Read the error message carefully — it usually tells you the line number and error type.
- 2. Use `print()` statements to inspect variable values at different points.
- 3. Test small pieces of code in isolation before combining them.
- 4. Use an IDE's built-in debugger to step through code line by line.
- 5. Never assume — verify your assumptions about what the code is doing.

Part B — Variables, Expressions, and Statements

2.1 Values and Data Types

A value is a basic piece of data that a program manipulates.

Value	Data type
20	Integer (int)
20.9	Floating point (float)
"welcome"	String (str)

```
print(type(17))
print(type("Hello"))
print(type(3.2))
```

Output:

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Strings

Strings can be enclosed in single quotes ('...'), double quotes ("..."), or triple quotes ("...", for multi-line strings).

```
'hello'      # single quotes → 'hello'
"hello"      # double quotes → 'hello'
'''hello
world'''     # triple quotes (multi-line) → 'hello\nworld'
```

Escape Sequences (used inside strings)

Escape sequence	Meaning
\n	Newline — moves to the next line
\t	Horizontal tab — inserts a tab space
\\	Backslash — prints a single backslash
\'	Single quote — used inside a single-quoted string
\"	Double quote — used inside a double-quoted string
\r	Carriage return — moves cursor to the start of the line

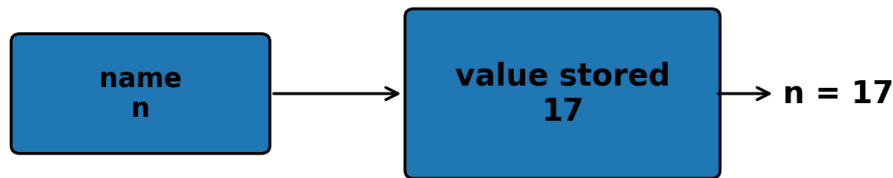
```
print("Name:\tRahul\nRoll No:\t07")
```

Output:

```
Name:  Rahul
Roll No:    07
```

2.2 Variables

How a Variable Works



A variable name refers to a value.

Figure: A variable name refers to a value stored in memory

A variable is a name that stores a value, allowing a program to remember and reuse data.

```
message = "Hello World"
n = 17
pi = 3.14159
```

Figure: Think of a variable as a labelled container — the name points to a value stored in memory (n = 17).

- Variables are mutable — their values can be changed.
- Use meaningful variable names for readability.
- The data type of a variable can change during execution (Python is dynamically typed).

Invalid: `17 = n` — because you can't assign a value directly to a literal.

Note: Read `"="` as "is assigned to", not "is equal to".

Mutable vs Immutable Data Types (preview)

Every Python value is either mutable (can be changed in place after creation) or immutable (cannot be changed — any 'change' actually creates a new value). This distinction becomes very important once you start using lists and dictionaries in later modules.

Immutable types	Mutable types
int, float, bool, str, tuple	list, dict, set
Cannot be changed after creation	Can be changed in place after creation
Example: <code>a = 5</code> then <code>a = 6</code> creates a new int object	Example: a list's elements can be updated without creating a new list

Sample Program — Swap Two Variables

Using a temporary variable:

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

temp = a
a = b
b = temp
print("After swap: a =", a, " b =", b)
```

Python's shortcut — without a temporary variable:

```
a, b = b, a # simultaneous (tuple) assignment
print("After swap: a =", a, " b =", b)
```

2.3 Variable Names and Keywords

Rules for naming variables:

- 1. It can contain letters, digits, and underscore (_).
- 2. Must start with a letter or underscore but never with a digit.
- 3. Names are case-sensitive: Score, score, SCORE are three different variables.
- 4. It cannot contain special characters like \$, #, @, *.
- 5. Keywords cannot be used as variable names.

Valid names	Invalid names	Reason invalid
num1	\$salary	\$ is not allowed
stdnt_rollno	1num	begins with a digit
my_name	stdnt name	contains a space
—	class	class is a reserved keyword

Keywords are reserved words with special meaning that form part of Python's syntax.

```
import keyword
print(keyword.kwlist)
```

Output:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
```

2.4 Statements

A statement is an instruction that the Python interpreter can execute.

```
x = 10 # 1. Assignment statement
if x > 0: print("positive") # 2. if statement
while x > 0: x = x - 1 # 3. loop statement
```

2.5 Evaluating Expressions

An expression is a combination of values, variables, operators, and functions that produces a result.

```
>>> 1 + 1
2
>>> len("Hello")
5
```

len() is a built-in function that returns the number of characters in a string.

Note: Evaluating an expression always produces a value — that's why expressions appear on the right-hand side (RHS) of assignment statements.

2.6 Operators and Operands

Operators are special tokens representing computations. Operands are the values an operator acts on.

Operator	Meaning	Expression	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division (float result)	5 / 3	1.666...
//	Floor division (int result)	5 // 3	1
**	Exponentiation	5 ** 3	125

```
n1 = int(input("Enter first number: "))
n2 = int(input("Enter second number: "))
print(n1 + n2)
print(n1 - n2)
print(n1 * n2)
print(n1 / n2)
print(n1 // n2)
print(n1 ** n2)
```

Note: input() returns a string by default — convert with int()/float() before doing arithmetic.

Comparison Operators (used in conditions & loops)

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 3	True
<	Less than	7 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	4 <= 3	False

Logical Operators (combine conditions)

Operator	Meaning	Example
and	True only if both conditions are true	(x > 0) and (x < 10)
or	True if at least one condition is true	(x < 0) or (x > 100)
not	Reverses/negates a boolean value	not (x == 5)

Memory tip: and needs BOTH to be happy; or is happy with just ONE; not simply flips True ↔ False.

2.7 Type Converter Functions

Function	Purpose	Example
int()	Converts to integer (truncates the decimal part)	int(3.99) → 3
float()	Converts to a floating-point number	float("77.86") → 77.86

Function	Purpose	Example
str()	Converts any type to a string	str(75) → '75'

```
>>> int(3.14)      # 3
>>> int(-3.999)    # -3   (truncates toward zero, does NOT round)
>>> int(99.999)    # 99   (does not round to nearest integer)
'>>> int('2346')    # 2346 (parses string → int)'
>>> int(21)        # 21   (works even if already an int)
>>> int('12xyz')   # ValueError: invalid literal for int()
```

Quick Reference: Handy Built-in Functions

Function	Purpose	Example
type()	Returns the data type of a value	type(5) → <class 'int'>
len()	Returns the number of characters/items	len("Hi") → 2
print()	Displays output on the screen	print("Hi")
input()	Reads a line of text from the user (always a string)	input()
abs()	Returns the absolute (positive) value	abs(-9) → 9
pow()	Raises a number to a power	pow(2,3) → 8
max() / min()	Returns the largest/smallest of the arguments	max(3,9,1) → 9
round()	Rounds a number to the nearest integer/decimal	round(3.567,1) → 3.6
range()	Generates a sequence of numbers (used with loops)	range(1,5) → 1,2,3,4

Sample Program — Temperature Converter (uses type converters)

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print("Temperature in Fahrenheit is", fahrenheit)
```

Output (for 37°C): Temperature in Fahrenheit is 98.6

2.8 Order of Operations

Operator precedence follows PEMDAS:

Letter	Meaning	Precedence
P	Parentheses	Highest
E	Exponentiation	Next highest
M, D	Multiplication, Division	Same level (left → right)
A, S	Addition, Subtraction	Same level (left → right)

Associativity: same-precedence operators evaluate left to right, except exponentiation, which is right to left.

```
5 + 4 * 3      # → 5 + 12 = 17
(5 + 4) * 3    # → 9 * 3 = 27
5 + 4 ** 2     # → 5 + 16 = 21
2 ** 3 ** 2    # → 2 ** 9 = 512   (right to left!)
(2 ** 3) ** 2  # → 8 ** 2 = 64
```

Tip: Always use parentheses to force the order you want and improve clarity.

Complete Operator Precedence Chart (highest → lowest)

Priority	Operators	Category
1 (highest)	()	Parentheses
2	**	Exponentiation
3	+x, -x	Unary plus / minus
4	*, /, //, %	Multiplication, division, floor division, modulus
5	+, -	Addition, subtraction
6	==, !=, >, <, >=, <=	Comparison operators
7	not	Logical NOT
8	and	Logical AND
9 (lowest)	or	Logical OR

Example combining arithmetic, comparison, and logical operators:

```
x = 5
print(x + 2 > 4 and x < 10)
# Step 1: x + 2 → 7
# Step 2: 7 > 4 → True
# Step 3: x < 10 → True
# Step 4: True and True → True
```

2.9 Operations on Strings

Invalid operations on strings:

```
message = "Hello"
message - 1          # Error
"Hello" + 123        # Error
"15" + 2             # Error – can't mix str and int with +
```

Concatenation (+): joins two strings end-to-end.

```
fruit = "banana"
baked_good = "nut bread"
print(fruit + baked_good)  # banananut bread
```

Repetition (*): repeats a string; one operand must be a string, the other an integer.

```
print("fun" * 3)  # funfunfun
```

2.10 The input() Function

input() gets input from the user and always returns a string.

```
name = input("Enter your name: ")
print("Hello", name)
```

Output:

```
Enter your name: Rahul
Hello Rahul
```

2.11 Composition

Programming languages let us take small building blocks and compose them into larger chunks.

```
# Step-by-step
response = input("What is your radius? ")
r = float(response)
area = 3.14159 * r ** 2
print("Area is", area)

# Single-line composition
r = float(input("What is your radius? "))
print("Area is", 3.14159 * r ** 2)
```

2.12 The Modulus Operator (%)

Gives the remainder when dividing two integers. Precedence similar to multiplication.

```
>>> q = 7 % 3
>>> print(q)
1                # remainder
>>> q = 7 / 3
>>> print(q)
2.333...         # quotient
```

Uses of the modulus operator:

- Checking divisibility (even/odd).
- Extracting the rightmost digit(s) of a number.
- Time conversion (seconds → hours/minutes/seconds).

Program — convert seconds into hours, minutes, and seconds:

```
total_secs = int(input("Enter total seconds: "))
hours = total_secs // 3600
secs_still_remaining = total_secs % 3600
minutes = secs_still_remaining // 60
secs_finally_remaining = secs_still_remaining % 60
print("Hrs =", hours, "Mins =", minutes, "Secs =", secs_finally_remaining)
```

Output (for 3666 seconds): Hrs = 1, Mins = 1, Secs = 6

Sample Program — Simple Interest Calculator

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter rate of interest (%): "))
time = float(input("Enter time period (years): "))
simple_interest = (principal * rate * time) / 100
print("Simple Interest is", simple_interest)
```

Output (for P=1000, R=5, T=2): Simple Interest is 100.0

Part C — Iteration

Computers excel at performing repetitive tasks without making mistakes. Iteration is the repeated execution of a set of statements, done via the for loop or the while loop.

3.1 Assignment & Updating Variables

```
num = 15
print(num)    # 15
```

Note: = is assignment, not equality. For comparison, use ==.

```
a = 7    # assignment
a == 7   # comparison, equality test
```

A variable can use its old value to compute a new one:

```
n = 5
n = 3 * n + 1    # n is now 16
```

Sample decision-making programs (positive/negative/zero, even/odd, largest of two/three numbers, power of a number) all follow the same if / elif / else pattern with comparison operators (>, <, ==, and).

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(num, "is even")
else:
    print(num, "is odd")
```

Program — extract ones, tens, hundreds place of an integer:

```
num = int(input("Enter an integer: "))
ones = num % 10
tens = (num // 10) % 10
hundreds = (num // 100) % 10
print("Ones:", ones, "Tens:", tens, "Hundreds:", hundreds)
```

Output (for 374): Ones = 4, Tens = 7, Hundreds = 3

Increment/Decrement shorthand:

```
x += 1    # same as x = x + 1
y -= 1    # same as y = y - 1
```

Sample Program — Check Leap Year (uses conditionals & modulus)

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a Leap Year")
else:
    print(year, "is Not a Leap Year")
```

Output (for 2024): 2024 is a Leap Year

3.2 The for Loop

Choosing a Loop

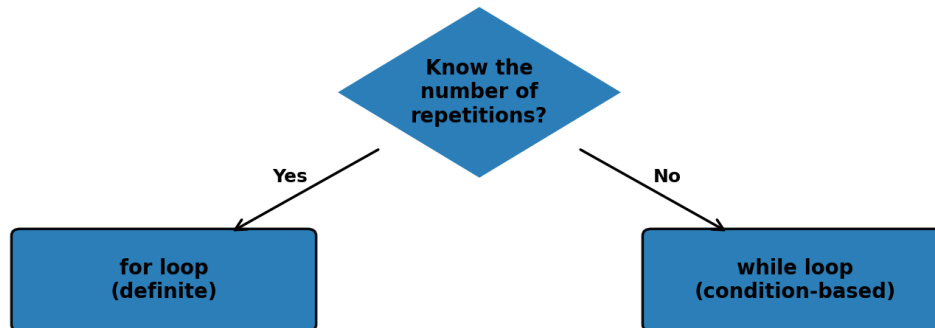


Figure: Choosing between for and while loops

Used to traverse each item in a sequence — ideal for definite iteration (number of repetitions known in advance).

```

numbers = [10, 20, 30, 40, 50]
total = 0
for num in numbers:
    total += num
print("The sum of 5 numbers is:", total)    # 150

# Sum of first N natural numbers
n = int(input("Enter N: "))
total = 0
for i in range(1, n + 1):
    total += i
print("Sum of first", n, "numbers is", total)
  
```

Real-world uses of for loops: processing every row in a spreadsheet/CSV, iterating over files in a folder, generating multiplication tables, validating each item in a list of student records.

The range() Function in Detail

range() generates a sequence of numbers and is most commonly used with for loops. It can take one, two, or three arguments:

Form	Meaning	Example	Generates
range(stop)	0 up to (not including) stop	range(5)	0, 1, 2, 3, 4
range(start, stop)	start up to (not including) stop	range(2, 6)	2, 3, 4, 5
range(start, stop, step)	start up to stop, in steps of step	range(1, 10, 2)	1, 3, 5, 7, 9
range(start, stop, -step)	counts downward	range(10, 0, -2)	10, 8, 6, 4, 2

```

for i in range(10, 0, -2):
    print(i, end=" ")
# Output: 10 8 6 4 2
  
```

Sample Program — Multiplication Table for a Given Number

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Output (for num=5): $5 \times 1 = 5$, $5 \times 2 = 10$, ... $5 \times 10 = 50$

3.3 The while Statement

Used for indefinite iteration — runs until a condition becomes false (number of iterations not known in advance).

Syntax:

```
while condition:
    # statements

n = 6
sum = 0
i = 0
while i <= n:
    sum += i
    i += 1
print("Sum is", sum)    # 21
```

Working of a while loop:

- 1. Evaluate the condition.
- 2. If True, execute the loop body.
- 3. Update the accumulator.
- 4. Update the loop-control variable so the condition eventually becomes False.
- 5. Repeat until False.

Note: If the condition never becomes false, you get an infinite loop.

for loop	while loop
Best for definite iteration (known count)	Best for indefinite iteration (unknown count)
Naturally iterates over a sequence	Needs manual condition & counter management
Lower risk of infinite loops	Higher risk of infinite loops if condition is mismanaged

3.4 The Collatz $3n + 1$ Sequence

Also called the Collatz conjecture — start with a positive integer n :

- If even: $n = n // 2$
- If odd: $n = 3n + 1$
- Repeat until n reaches 1.

```
n = int(input("Enter any value for n: "))
steps = 0
while n != 1:
    print(n, end=", ")
    if n % 2 == 0:
        n = n // 2
    else:
```

```
n = 3 * n + 1
steps += 1
print(n)
print("Total steps taken:", steps)
```

Output (n=12): 12, 6, 3, 10, 5, 16, 8, 4, 2, 1 → Total steps taken: 9

Fun fact: The Collatz conjecture is unsolved in mathematics — it's conjectured that every positive integer eventually reaches 1, but this has never been formally proven for all numbers.

3.5 Tables (1D and 2D)

Loops can systematically generate tables.

```
for x in range(13):
    print(x, "\t", 2 ** x)    # powers of 2, from 0 to 12
```

2D table example — first six multiples of 2:

```
for i in range(1, 7):
    print(2 * i, end=" ")
print()
```

Output: 2 4 6 8 10 12

Note: end controls what's printed at the end of each print() call (default is a newline).

Sample Program — Right-Angled Triangle Star Pattern (nested loops)

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    for j in range(i):
        print("*", end="")
    print()
```

Output (for rows=4):

```
*
**
***
****
```

3.6 break and continue

Statement	Effect
break	Immediately exits the loop, even if the condition hasn't finished
continue	Skips the rest of the current iteration and moves to the next one

```
# break example
for i in [12, 14, 17, 22, 19]:
    if i % 2 == 1:
        break
    print(i)
print("done")
# Output: 12, 14, done

# continue example
for i in [12, 16, 17, 24, 29, 30]:
    if i % 2 == 1:
        continue
```

```
print(i)
print("done")
# Output: 12, 16, 24, 30, done
```

The pass Statement (bonus)

pass is a null operation — it does absolutely nothing when executed. It's used as a placeholder where Python's syntax requires a statement, but you don't want any code to run yet (e.g., while sketching out a program's structure).

```
for i in [12, 14, 17, 22, 19]:
    if i % 2 == 1:
        pass    # TODO: handle odd numbers later
    else:
        print(i)
```

Memory tip: break exits, continue skips, pass does nothing (it's just a placeholder).

3.7 Paired Data & Nested Loops for Nested Data

Related information can be grouped into tuples:

```
year_born = ("Amal", 1990)
celebs = [("Amal", 1990), ("Akbar", 1993), ("Anthony", 1985)]
for name, year in celebs:
    if year < 1990:
        print(name)    # Anthony
```

Nested data — name plus a list of subjects:

```
students = [
    ("John", ["CompSci", "Physics"]),
    ("Vusi", ["Maths", "CompSci", "Stats"]),
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
]
count = 0
for name, subjects in students:
    for s in subjects:
        if s == "CompSci":
            count += 1
print("The number of students taking CompSci is", count)    # 3
```

Shortcut: replace the inner loop with: if "CompSci" in subjects: count += 1

Part D — Functions

4.1 Functions with Arguments & Return Values

Function Thinking

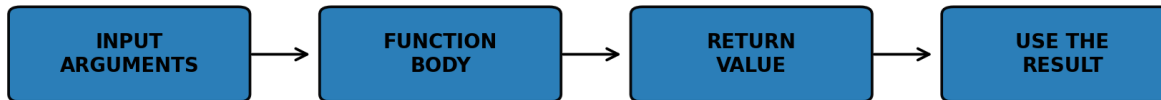


Figure: Arguments enter a function; a return value carries the result out

A function is a named sequence of statements. It organizes a program into logical chunks that match how we think about the problem.

Syntax:

```
def function_name(parameters):
    statements

abs(5)           # returns 5
abs(-5)          # returns 5
pow(2, 3)        # returns 8
max(4, 7, 1, 2, 17) # returns 17
```

Fruitful vs. void functions:

Type	Description	Example
Fruitful function	Computes and sends back a result via return	max(), abs(), a custom find_average()
Void function	Performs an action but returns nothing (None by default)	print()

Program — calculate the average of three numbers using a user-defined function:

```
def find_average(num1, num2, num3):
    avg = (num1 + num2 + num3) / 3
    return avg

n1 = int(input("Enter first number: "))
n2 = int(input("Enter second number: "))
n3 = int(input("Enter third number: "))
result = find_average(n1, n2, n3)
print("The average is:", result)
```

Output:

```
Enter first number: 20
Enter second number: 20
Enter third number: 18
The average is: 19.333333333333332
```

Parameters vs. Arguments:

Term	Meaning	In the example above
Parameters	Names used inside the function definition	num1, num2, num3
Arguments	The actual values passed when calling the function	n1, n2, n3

Advantages of using functions

- 1. Reusability — write once, call many times.
- 2. Modularity — breaks a big problem into smaller, manageable pieces.
- 3. Readability — a well-named function documents itself.
- 4. Easier debugging & testing — isolate and test each function independently.
- 5. Abstraction — the caller doesn't need to know how a function works internally, only what it does.

Common pitfalls

- Forgetting a return statement (the function then returns None by default).
- Confusing parameters with arguments.
- Modifying a variable inside a function without returning/using the result.

Part E — Practice Programs (Solved Examples)

These extra solved programs combine loops, conditionals, and functions from this module — good practice before lab exams and vivas.

5.1 Factorial of a Number (using a loop)

```
num = int(input("Enter a number: "))
fact = 1
for i in range(1, num + 1):
    fact *= i
print("Factorial is", fact)
```

Output (for 5): Factorial is 120

5.2 Fibonacci Sequence (first N terms)

```
n = int(input("Enter number of terms: "))
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output (for n=6): 0 1 1 2 3 5

Note: `a, b = b, a + b` is Python's simultaneous (tuple) assignment — both values are updated together in a single step.

5.3 Check Whether a Number Is Prime

```
num = int(input("Enter a number: "))
is_prime = True
if num < 2:
    is_prime = False
else:
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
if is_prime:
    print(num, "is Prime")
else:
    print(num, "is Not Prime")
```

Memory tip: you only need to test divisors up to $\sqrt{\text{num}}$ — if no factor exists below that, none exists above it either.

5.4 Reverse the Digits of a Number

```
num = int(input("Enter a number: "))
reverse = 0
temp = num
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp = temp // 10
print("Reversed number is", reverse)
```

Output (for 374): Reversed number is 473

5.6 Senior Citizen Check (Name & Year of Birth)

```
name = input("Enter your name: ")
birth_year = int(input("Enter your year of birth: "))
current_year = 2026
age = current_year - birth_year
if age >= 60:
    print(name, "is a Senior Citizen")
else:
    print(name, "is not a Senior Citizen")
```

Output (for birth_year=1955): is a Senior Citizen (age 71)

5.7 Mean, Variance & Standard Deviation of N Numbers

This combines a for loop (Part C) with expressions and the exponent operator (Part B).

```
n = int(input("Enter how many numbers: "))
numbers = []
for i in range(n):
    val = float(input("Enter number: "))
    numbers.append(val)

mean = sum(numbers) / n
squared_diff_sum = 0
for x in numbers:
    squared_diff_sum += (x - mean) ** 2
variance = squared_diff_sum / n
std_dev = variance ** 0.5 # square root via exponent 0.5

print("Mean:", mean)
print("Variance:", variance)
print("Standard Deviation:", std_dev)
```

Note: `sum()` and `append()` are quick previews of built-in/list features covered fully in Module 2 — but the loop logic itself is pure Module 1.

5.8 Frequency of Each Digit in a Number

```
num_str = input("Enter a multi-digit number: ")
for digit in "0123456789":
    count = 0
    for ch in num_str:
        if ch == digit:
            count += 1
    if count > 0:
        print("Digit", digit, "occurs", count, "time(s)")
```

Output (for 112233): Digit 1 occurs 2 time(s), Digit 2 occurs 2 time(s), Digit 3 occurs 2 time(s)

5.9 Armstrong Number Check

An Armstrong number equals the sum of its own digits, each raised to the power of the number of digits (e.g., $153 = 1^3 + 5^3 + 3^3$).

```
num = int(input("Enter a number: "))
temp = num
num_digits = len(str(num))
total = 0
while temp > 0:
```

```

digit = temp % 10
total += digit ** num_digits
temp = temp // 10
if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")

```

5.10 GCD (HCF) of Two Numbers

```

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
gcd = 1
for i in range(1, min(a, b) + 1):
    if a % i == 0 and b % i == 0:
        gcd = i
print("GCD is", gcd)

```

5.11 Palindrome Number Check

A number is a palindrome if it reads the same forwards and backwards — this reuses the digit-reversal logic from Section 5.4.

```

num = int(input("Enter a number: "))
temp = num
reverse = 0
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp = temp // 10
if reverse == num:
    print(num, "is a Palindrome")
else:
    print(num, "is Not a Palindrome")

```

5.12 Menu-Driven Function for Basic Arithmetic

A common lab exercise: read 2 numbers and a choice, then call a function to perform the chosen operation.

```

def calculate(n1, n2, choice):
    if choice == 1:
        return n1 + n2
    elif choice == 2:
        return n1 - n2
    elif choice == 3:
        return n1 * n2
    elif choice == 4:
        return n1 / n2
    else:
        return None

n1 = float(input("Enter first number: "))
n2 = float(input("Enter second number: "))
print("1-Add 2-Subtract 3-Multiply 4-Divide")
choice = int(input("Enter your choice: "))
result = calculate(n1, n2, choice)
if result is not None:
    print("Result is", result)

```

```
else:  
    print("Invalid choice")
```

Appendix

Appendix A — Glossary of Key Terms

Term	Meaning
Interpreter	Program that translates and executes source code line by line
Bytecode	Intermediate, platform-independent code (.pyc) that the Python Virtual Machine executes
Syntax	The set of grammar rules that define valid code structure
Bug	A mistake in a program that causes incorrect or unexpected behavior
Exception	An error detected during execution (a runtime error)
Literal	A fixed value written directly in code, e.g. 5, "hi", 3.14
Iterable	Any object (like a list, string, or range) that can be looped over
Namespace	A container that holds a mapping of variable names to values
Argument	The actual value passed into a function when it is called
Parameter	The variable name listed in a function's definition

Appendix B — Do's and Don'ts for Writing Good Python Code

Do

- Use meaningful, descriptive variable and function names (e.g., `student_marks`, not `sm`).
- Follow `snake_case` for variable/function names (Python convention, part of PEP 8).
- Add comments (`#`) to explain why the code does something, not just what it does.
- Keep indentation consistent — 4 spaces per level.
- Break large problems into small, well-named functions.
- Test your code with edge cases (0, negative numbers, empty input).

Don't

- Don't use single-letter or vague names for anything but loop counters (`i`, `j`).
- Don't hard-code 'magic numbers' without explanation — use named constants instead.
- Don't mix tabs and spaces for indentation.
- Don't write overly long functions that do many unrelated things.
- Don't ignore error messages — read them, they usually point straight at the bug.

Appendix C — Quick-Revision Summary

Concept	Key takeaway
Program	Sequence of instructions for a computation
Debugging	Finding & fixing bugs — syntax, runtime, semantic errors
Algorithm / Flowchart / Pseudocode	Three problem-solving tools; each trades off precision vs. visual clarity

Concept	Key takeaway
Variable	Named storage for a value; mutable, dynamically typed
Expression	Combination of values/operators/functions producing a result
Operator precedence	PEMDAS — parentheses first, exponents right-to-left, then $*$ /, then $+$ -
Modulus %	Gives the remainder; used for divisibility, digit extraction, time conversion
for loop	Definite iteration — traverses a sequence
while loop	Indefinite iteration — runs until a condition is false
break	Exits the loop immediately
continue	Skips to the next iteration
Tuple	Groups related, ordered data into pairs
Function	Reusable named block; takes parameters, may return a value

MODULE 1 — PRACTICE QUESTION BANK

100 Practice Questions • Based on the Complete Module 1 Notes

Prof. Kiran Mudaraddi

How to use these questions: Use them for revision, classroom discussion, self-practice, assignments, viva preparation, and formative assessment. The interactive 20-MCQ quiz remains on the website and is intentionally not reproduced here.

Part A — Programming Fundamentals

1. What is a program?
2. What is meant by a sequence of instructions in programming?
3. List the basic building blocks of a program.
4. What is meant by input in a program?
5. What is meant by output in a program?
6. What is conditional execution?
7. What is repetition in a program?
8. What is mathematical computation? Give one example.
9. What is symbolic computation? Give one example.
10. What is a high-level programming language?
11. What is a low-level programming language?
12. How does assembly language differ from a high-level language?
13. State two advantages of high-level programming languages.
14. What are the three main problem-solving tools introduced in Module 1?
15. List the characteristics of a good algorithm.
16. What is a flowchart?
17. What is a syntax error?
18. What is a runtime error?
19. What is a semantic error?
20. What is experimental debugging?

Part B — Variables, Expressions & Statements

21. What is a value in Python?
22. Name three basic Python data types discussed in Module 1.
23. What data type is the value 20?
24. What data type is the value 20.9?
25. What data type is the value "welcome"?
26. What is a string?
27. What are the three ways of writing strings shown in the notes?
28. What is the purpose of triple-quoted strings?
29. What does the escape sequence `\n` do?
30. What does the escape sequence `\t` do?
31. What does the escape sequence `\\` represent?
32. What is a variable?

33. Why are meaningful variable names recommended?
34. What does assignment using = mean in Python?
35. Why is 17 = n invalid?
36. What does dynamically typed mean in the context of Python variables?
37. What is the difference between mutable and immutable values?
38. Name two immutable Python types listed in the notes.
39. Name two mutable Python types listed in the notes.
40. What is a keyword in Python?
41. Why cannot class be used as an ordinary variable name?
42. State two rules for naming Python variables.
43. What is a statement?
44. What is an expression?
45. What does evaluating an expression produce?
46. What do and, or, and not do?
47. What does int() do?
48. What does float() do?
49. What does str() do?
50. Why does input() usually need a type conversion before arithmetic?
51. What is composition in programming?
52. What does the modulus operator % return?
53. Give two uses of the modulus operator.
54. How can % be used to check whether an integer is even?
55. How can % be used to extract the rightmost digit?
56. What is the difference between = and ==?
57. How can a variable use its old value to compute a new value?
58. What does x += 1 mean?
59. What does y -= 1 mean?
60. What is iteration?
61. When is a for loop especially useful?
62. What does a for loop naturally traverse?
63. What is range(stop)?
64. What is range(start, stop)?
65. What is range(start, stop, step)?

Part C — Iteration

66. What does range(10, 0, -2) generate?
67. Why is the stop value of range() not included?
68. What is a while statement used for?
69. What is the basic execution cycle of a while loop?
70. Why must a while loop update its loop-control variable when necessary?
71. What is an infinite loop?
72. State one difference between for and while loops.

- 73. What is the Collatz $3n + 1$ sequence?
- 74. What happens to n in the Collatz sequence when n is even?
- 75. What happens to n in the Collatz sequence when n is odd?
- 76. When does the Collatz loop in the notes stop?
- 77. For $n = 12$, what sequence is produced by the Collatz example?
- 78. What are tables useful for when using loops?
- 79. What does the end argument of `print()` control?
- 80. What is meant by a two-dimensional table in the context of nested iteration?
- 81. What is a nested loop?
- 82. In a star-pattern program, what can the outer loop represent?
- 83. In a star-pattern program, what can the inner loop represent?
- 84. What does `break` do inside a loop?
- 85. What does `continue` do inside a loop?
- 86. What is `pass` used for?
- 87. What is paired data?
- 88. How can tuples be used to group paired information?
- 89. What is nested data?
- 90. How do nested loops process nested data?

Part D — Functions

- 91. What is a function?
- 92. Why are functions useful in a program?
- 93. What is the syntax for defining a function?
- 94. What is a parameter?
- 95. What is an argument?
- 96. What is the difference between a parameter and an argument?
- 97. What is a return value?
- 98. What is a fruitful function?
- 99. What is a void function in the terminology used in the notes?
- 100. What happens when a function has no return statement?

THANK YOU

Part E — Practice Programs (Solved Examples)

The following practice set follows the program list used in the Module 1 website. Each program is presented with a short purpose, Python code, and sample output. These examples reinforce variables, expressions, conditionals, loops, strings, functions, and problem-solving.

Practice Program Set

No.	Program
1	Swap Two Variables
2	Temperature Converter
3	Simple Interest Calculator
4	Leap Year Check
5	Multiplication Table
6	Star Pattern
7	Factorial of a Number
8	Fibonacci Sequence
9	Prime Number Check
10	Reverse Digits
11	Senior Citizen Check
12	Mean, Variance and Standard Deviation
13	Frequency of Each Digit
14	Armstrong Number Check
15	GCD (HCF) of Two Numbers
16	Palindrome Number Check
17	Menu-Driven Calculator using a Function

1. Swap Two Variables

Purpose: Exchange the values of two variables using Python assignment.

Python Program

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
a, b = b, a
print("After swap: a =", a, "b =", b)
```

Sample Output

```
Enter a: 10
Enter b: 20
After swap: a = 20 b = 10
```

2. Temperature Converter

Purpose: Convert Celsius temperature to Fahrenheit using an arithmetic expression.

Python Program

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print("Temperature in Fahrenheit is", fahrenheit)
```

Sample Output

```
Enter temperature in Celsius: 37
Temperature in Fahrenheit is 98.6
```

3. Simple Interest Calculator

Purpose: Calculate simple interest from principal, rate, and time.

Python Program

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter rate of interest (%): "))
time = float(input("Enter time period (years): "))
simple_interest = (principal * rate * time) / 100
print("Simple Interest is", simple_interest)
```

Sample Output

```
Enter principal amount: 1000
Enter rate of interest (%): 5
Enter time period (years): 2
Simple Interest is 100.0
```

4. Leap Year Check

Purpose: Check a year using the standard divisibility conditions.

Python Program

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a Leap Year")
else:
    print(year, "is Not a Leap Year")
```

Sample Output

```
Enter a year: 2024
2024 is a Leap Year
```

5. Multiplication Table

Purpose: Print the multiplication table of a number using a for loop.

Python Program

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Sample Output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

6. Star Pattern

Purpose: Use nested loops to print an increasing right-triangle pattern.

Python Program

```
n = int(input("Enter number of rows: "))
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Sample Output

```
Enter number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

7. Factorial of a Number

Purpose: Calculate $n!$ using a loop.**Python Program**

```
num = int(input("Enter a number: "))
fact = 1
for i in range(1, num + 1):
    fact *= i
print("Factorial is", fact)
```

Sample Output

```
Enter a number: 5
Factorial is 120
```

8. Fibonacci Sequence

Purpose: Display the first N terms of the Fibonacci sequence.**Python Program**

```
n = int(input("Enter number of terms: "))
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Sample Output

```
Enter number of terms: 6
0 1 1 2 3 5
```

9. Prime Number Check

Purpose: Determine whether a number is prime by testing possible divisors.**Python Program**

```
num = int(input("Enter a number: "))
is_prime = True
if num < 2:
    is_prime = False
else:
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
if is_prime:
```

```

    print(num, "is Prime")
else:
    print(num, "is Not Prime")

```

Sample Output

```

Enter a number: 29
29 is Prime

```

10. Reverse Digits

Purpose: Reverse the digits of an integer using modulus and floor division.**Python Program**

```

num = int(input("Enter a number: "))
reverse = 0
temp = num
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp = temp // 10
print("Reversed number is", reverse)

```

Sample Output

```

Enter a number: 374
Reversed number is 473

```

11. Senior Citizen Check

Purpose: Calculate age from the year of birth and check whether the person is 60 or older.**Python Program**

```

name = input("Enter your name: ")
birth_year = int(input("Enter your year of birth: "))
current_year = 2026
age = current_year - birth_year
if age >= 60:
    print(name, "is a Senior Citizen")
else:
    print(name, "is not a Senior Citizen")

```

Sample Output

```

Enter your name: Ravi
Enter your year of birth: 1955
Ravi is a Senior Citizen

```

12. Mean, Variance and Standard Deviation

Purpose: Read N numbers and compute the mean, variance, and standard deviation.**Python Program**

```

n = int(input("Enter how many numbers: "))
numbers = []
for i in range(n):
    val = float(input("Enter number: "))
    numbers.append(val)
mean = sum(numbers) / n
squared_diff_sum = 0
for x in numbers:
    squared_diff_sum += (x - mean) ** 2

```

```

variance = squared_diff_sum / n
std_dev = variance ** 0.5
print("Mean:", mean)
print("Variance:", variance)
print("Standard Deviation:", std_dev)

```

Sample Output

```

Enter how many numbers: 3
Enter number: 10
Enter number: 20
Enter number: 30
Mean: 20.0
Variance: 66.66666666666667
Standard Deviation: 8.16496580927726

```

13. Frequency of Each Digit

Purpose: Count how many times each digit occurs in a number.

Python Program

```

num_str = input("Enter a multi-digit number: ")
for digit in "0123456789":
    count = 0
    for ch in num_str:
        if ch == digit:
            count += 1
    if count > 0:
        print("Digit", digit, "occurs", count, "time(s)")

```

Sample Output

```

Enter a multi-digit number: 112233
Digit 1 occurs 2 time(s)
Digit 2 occurs 2 time(s)
Digit 3 occurs 2 time(s)

```

14. Armstrong Number Check

Purpose: Check whether a number equals the sum of its digits raised to the number of digits.

Python Program

```

num = int(input("Enter a number: "))
temp = num
num_digits = len(str(num))
total = 0
while temp > 0:
    digit = temp % 10
    total += digit ** num_digits
    temp = temp // 10
if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")

```

Sample Output

```

Enter a number: 153
153 is an Armstrong number

```

15. GCD (HCF) of Two Numbers

Purpose: Find the greatest common divisor using repeated divisibility checks.

Python Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
gcd = 1
for i in range(1, min(a, b) + 1):
    if a % i == 0 and b % i == 0:
        gcd = i
print("GCD is", gcd)
```

Sample Output

```
Enter first number: 24
Enter second number: 18
GCD is 6
```

16. Palindrome Number Check

Purpose: Check whether a number reads the same forwards and backwards.

Python Program

```
num = int(input("Enter a number: "))
temp = num
reverse = 0
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp = temp // 10
if reverse == num:
    print(num, "is a Palindrome")
else:
    print(num, "is Not a Palindrome")
```

Sample Output

```
Enter a number: 121
121 is a Palindrome
```

17. Menu-Driven Calculator using a Function

Purpose: Read two numbers and a choice, then call a function for the selected arithmetic operation.

Python Program

```
def calculate(n1, n2, choice):
    if choice == 1:
        return n1 + n2
    elif choice == 2:
        return n1 - n2
    elif choice == 3:
        return n1 * n2
    elif choice == 4:
        return n1 / n2
    else:
        return None

n1 = float(input("Enter first number: "))
n2 = float(input("Enter second number: "))
print("1-Add 2-Subtract 3-Multiply 4-Divide")
choice = int(input("Enter your choice: "))
result = calculate(n1, n2, choice)
```

```
if result is not None:  
    print("Result is", result)  
else:  
    print("Invalid choice")
```

Sample Output

```
Enter first number: 10  
Enter second number: 5  
1-Add 2-Subtract 3-Multiply 4-Divide  
Enter your choice: 1  
Result is 15.0
```

THANK YOU

Keep learning • Keep coding • Keep debugging
Python Programming — Module 1
Prof. Kiran Mudaraddi